

05-src-features

05 — src features extraction

Exhaustive extraction of the reference BobaLink browser-extension feature layer (the Browser Torrent Extension Guide/src/src tree). This is the implementation the Boba plan must reproduce and integrate with the 7186 (download-proxy) / 7187 (merge service) / 7189 (boba-jackett) services. Every signature, endpoint, message type, storage key, and default below is cited to the file it lives in.

§11.4.10 secrets flag (read first): the reference code ships a **HARD-CODED fixed passphrase** "bobalink-extension" used to AES-encrypt every stored credential (options/options.ts:327). That is effectively no protection — anyone with the source can decrypt chrome.storage.local. Cookie-auth sendTorrents also passes an **empty passphrase** (background/index.ts:409), which can only ever decrypt creds that were encrypted with "". Both are leak vectors the Boba port MUST replace (see Open Questions).

Files read

Prompt-specified files (read in full)

File	Lines
api/client.ts	565
api/auth.ts	364
api/qbittorrent.ts	247
api/queue.ts	385
api/health.ts	288
parser/bencode.ts	427
parser/magnet.ts	338
parser/torrent-file.ts	357
scanner/base.ts	274

File	Lines
scanner/orchestrator.ts	401
scanner/link-scanner.ts	253
scanner/text-scanner.ts	134
scanner/site-db.ts	326
content/index.ts	306
content/scanner.ts	90
content/highlight.ts	252
content/styles.css	87
background/index.ts	759
popup/popup.ts	496
popup/index.html	69
popup/styles.css	418
options/options.ts	710
options/index.html	292
options/styles.css	761 (not opened — pure presentation, not load-bearing for the plan)
assets/icon.svg	25

Supporting files also read (referenced by the above; load-bearing for keys/defaults/types)

File	Lines	Why
shared/constants.ts	403	endpoints, storage keys, regexes, ports, retry/rate/timeout config, site selectors, badge colors
shared/crypto.ts	292	AES-256-GCM / PBKDF2 credential encryption (§11.4.10)
shared/storage.ts	248	chrome.storage.local wrapper + change listener
shared/utils.ts	366	debounce, TokenBucket, retryWithBackoff, getDomain, isValidHttpUrl
shared/events.ts	175	

File	Lines	Why
types/config.ts	232	TypedEventEmitter + EventMap ServerConfig, ExtensionConfig, DEFAULT_CONFIG
types/api.ts	486	qBt types, AuthCredentials/ AuthState, QueueItem, MessageType, ExtensionMessage
types/torrent.ts	223	MagnetInfo, TorrentFile, ParsedTorrent, DetectedTorrent, SendResult, PageScanResult

shared/logger.ts and shared/errors.ts exist (imported widely: createLogger, ParseError, NetworkError, ServerError, RateLimitError, AuthError, ConfigError, TorrentError, StorageError, normalizeError) but were not in the read scope; their behavior is inferred from call-sites (typed error classes with getUserMessage(), cause, and context fields).

1. Manifest (what this extension is)

A **Manifest V3** Chrome/Chromium extension named **BobaLink** (EXT.NAME, constants.ts) that scans web pages for **magnet links** and **.torrent files**, then sends them to a **qBittorrent WebUI** (or a Boba backend exposing the same WebUI API). Architecture:

- **Background service worker** (background/index.ts) — central message router / hub.
- **Content script** (content/*) — runs on every page, scans DOM, highlights, reports up.
- **Popup UI** (popup/*) — per-tab torrent list + send button + connection status.
- **Options UI** (options/*) — server config, auth, general/advanced settings.
- **API layer** (api/*) — qBittorrent WebUI client, auth, offline queue, health.
- **Parsers** (parser/*) — bencode, magnet URI, .torrent file + infohash (SHA-1).
- **Scanners** (scanner/*) — orchestrator + link/text scanners + known-site DB.

Build stack (per options/index.html About): Manifest V3 service worker, TypeScript 5.7, WXT build system, AES-256-GCM credential encryption. Version v1.0.0.

2. API client surface, auth, retry & queue mechanics

2.1 BobaAPIClient (api/client.ts)

The low-level HTTP client. **Base URL** is passed to the constructor and normalized by stripping a trailing slash (client.ts:83). Talks to qBittorrent's **WebUI v2 API** (/api/v2/...). All endpoint path constants live in QBITTORRENT_ENDPOINTS (constants.ts:109).

Construction - constructor(baseUrl: string, requestTimeout = REQUEST_TIMEOUTS.DEFAULT /*15000ms*/) — builds a TokenBucket(MAX_REQUESTS=10, refillRate=10/(1000/1000)=10 tok/s) rate limiter (client.ts:81-88, RATE_LIMIT in constants.ts:198).

Auth / header state - setAuthCookie(cookie: string | null): void — stores the qBittorrent SID value; sent as Cookie: SID=<value> header on every request (client.ts:98, client.ts:449-451). - setHeaders(headers: Record<string,string>): void — custom headers (used for X-API-Key and Authorization: Basic ...) (client.ts:108). - getBaseUrl(): string (client.ts:117).

Auth endpoints - login(username, password): Promise<boolean> — POST /api/v2/auth/login (QBITTORRENT_ENDPOINTS.AUTH_LOGIN), body = FormData{username,password}, timeout REQUEST_TIMEOUTS.AUTH (10000ms). On HTTP 200 it extracts SID from the set-cookie header via regex /SID=([^\;]+)/; if absent, falls back to checking the response text for "ok". Returns false on any non-200 or thrown error (caught internally) (client.ts:128-166). - Note: browsers normally hide set-cookie from fetch; this relies on credentials:"include" and works in practice because the cookie is also set on the jar, but the explicit SID regex parse is fragile (Open Question). - logout(): Promise<void> — POST /api/v2/auth/logout with retry:false; clears the cookie (client.ts:171-179).

App / torrent endpoints

Method	Verb + path	Payload
getVersion(): Promise<string>	GET /api/v2/app/version	—

Method	Verb + path	Payload
<code>addTorrentFromMagnet(magnetUri, options?)</code>	POST /api/v2/torrents/add	FormData{urls:<magnet>, ...addOpts}
<code>addTorrentFromFile(file: File, options?)</code>	POST /api/v2/torrents/add	FormData{torrents:<File>, ...addOpts}
<code>getTorrents(filter?)</code>	GET /api/v2/torrents/info?filter=<f>	—
<code>deleteTorrents(hashes[], deleteFiles=false)</code>	POST /api/v2/torrents/delete	FormData{hashes:h1\ h2, deleteFiles:"true"/"false"}
<code>pauseTorrents(hashes[])</code>	POST /api/v2/torrents/pause	FormData{hashes:h1\ h2}
<code>resumeTorrents(hashes[])</code>	POST /api/v2/torrents/resume	FormData{hashes:h1\ h2}

Defined-but-unused endpoint constants: APP_PREFERENCES, APP_SET_PREFERENCES, TRANSFER_INFO (constants.ts:116-127) — no client method calls them yet.

applyAddOptions(formData, options) (client.ts:373-392) maps QBittorrentAddTorrentParams fields onto the FormData: savepath, category, tags, skip_checking, paused, root_folder, rename, upLimit, dlLimit, autoTMM, contentLayout, sequentialDownload, firstLastPiecePrio.

Generic verbs: get<T>(path,opts), post<T>(path,body,opts), delete<T>(path,opts) — each calls requestWithRetry then parseResponse<T> (client.ts:330-365).

Retry / backoff / timeout / rate limiting - requestWithRetry (client.ts:402): if options.retry !== false, wraps requestRaw in retryWithBackoff(fn, maxRetries=RETRY_CONFIG.MAX_RETRIES /*3*/, base=1000ms, max=30000ms). - retryWithBackoff (utils.ts:168):

attempts 0..maxRetries; delay = min(base*2^attempt, max) + random*0.3*clampedDelay (exponential + 30% jitter). Throws the last error after exhausting retries. - requestRaw (client.ts:430): - **Rate limit:** rateLimiter.consume() (TokenBucket). If no token, sleeps 1000 / availableTokens ms then proceeds (it does NOT re-check — soft limiter) (client.ts:436-440). - **Timeout:** AbortController + setTimeout(abort, timeout); on abort throws NetworkError("Request timeout after <ms>ms") (client.ts:470-497). - **Headers:** Cookie: SID=... if set, then custom headers, then per-request headers. Content-Type: application/x-www-form-urlencoded is set ONLY when body is not FormData (browser sets multipart boundary for FormData) (client.ts:445-468). - fetch(url, {method, headers, body, signal, credentials:"include"}) (client.ts:474). - On !response.ok → handleErrorResponse. - handleErrorResponse (client.ts:542): HTTP **429** → reads retry-after header (seconds × 1000, default 60000) → throws RateLimitError(msg, retryMs). Other non-OK → reads body text → throws ServerError(message, status). - parseResponse<T> (client.ts:512): 204 or content-length:"0" → undefined. text/plain content-type → raw text. Else tries response.json(), falling back to text.

2.2 Auth flow AuthHandler (api/auth.ts) — §11.4.10 relevant

Four auth methods (AuthMethod = "none" | "cookie" | "api_key" | "basic", config.ts:13):

- constructor(client, initialMethod="none") builds an initial AuthState (all-null, isAuthenticated:false, consecutiveFailures:0) (auth.ts:44, createInitialState 352).
- authenticate(credentials: AuthCredentials): Promise<boolean> (auth.ts:73) dispatches:
 - cookie → authenticateCookie(user,pass) → client.login(); on success records sidCookie (read from client["authCookie"] private field — a TS escape hatch, auth.ts:247) + sidExpiresAt = now + 3600_000.
 - api_key → authenticateApiKey(apiKey) → client.setHeaders({"X-API-Key": apiKey}); **always returns true** (validated on first real request); stores a redacted apiKeyHeader: "X-API-Key xxxx..." (only first 4 chars) (auth.ts:261).
 - basic → authenticateBasic(user,pass) → btoa(user:pass) → client.setHeaders({"Authorization":"Basic <b64>"}); always true; stores redacted basicAuthHeader (first 8 b64 chars) (auth.ts:284).
 - none → authenticateNone() → just calls client.getVersion() to confirm reachability (auth.ts:307).
 - On success: sets isAuthenticated:true, lastRefreshedAt:now, consecutiveFailures:0.

- On failure: `recordFailure(msg)` increments `consecutiveFailures`; at `MAX_CONSECUTIVE_FAILURES = 3` throws `AuthError (auth.ts:28, 327)`.
- `logout()`: calls `client.logout()`, then resets state + clears cookie + clears headers (`auth.ts:122`).
- `refreshIfNeeded(credentials)`: `api_key/basic/none` never expire (returns current `isAuthenticated`). Cookie: re-authenticates if not authed, or if `lastRefreshedAt` older than `COOKIE_LIFETIME_MS = 3600_000` (`auth.ts:142`).
- **static `createCredentialsFromConfig(config, passphrase)`** (`auth.ts:170`) — **decrypt point**: dynamically imports `shared/crypto`, parses `config.encryptedPassword / encryptedApiKey` (JSON `EncryptedBundle`) and `decrypt(bundle, passphrase)`s it to build the live `AuthCredentials`. Throws `ConfigError` if required fields are missing.
- `getState()`, `isAuthenticated()` accessors.

Crypto (shared/crypto.ts) — AES-256-GCM, key derived per-encryption via PBKDF2 (`ENCRYPTION` in `constants.ts:246`: AES-GCM, 256-bit key, 12-byte IV, 16-byte salt, PBKDF2 SHA-256, **100000 iterations**, key version 1). `encrypt(plaintext, passphrase)` returns `EncryptedBundle{salt,iv,ciphertext,version}` (all base64); `decrypt(bundle, passphrase)` reverses it. Also `sha256(str)`, `simpleHash(str)`, `isEncrypted(v)`, `generateSecurePassphrase()` (32 random bytes, **unused** by the options flow which uses the fixed string instead). The 128-bit GCM tag is appended to ciphertext.

2.3 Offline queue `OfflineQueue (api/queue.ts)`

Persistent retry queue for failed sends, stored under `STORAGE_KEYS.QUEUE ("bobalink_queue")`. Constants: `DEFAULT_MAX_SIZE=50`, `PROCESS_INTERVAL_MS=60000`, `ITEM_SEND_DELAY_MS=500` (`queue.ts:24-34`).

- `init()` — loads persisted `QueueItem[]` from storage (`queue.ts:62`).
- `enqueue(infohash, magnetUri, torrentUrl, displayName, serverId, priority="normal")`: `Promise<QueueItem>` — evicts oldest while at/over `maxSize` (FIFO drop), builds a `QueueItem{id, torrent{infohash,magnetUri,torrentUrl,displayName}, serverId, addedAt, attempts:0, lastError:null, lastAttemptAt:null, priority}`, pushes, persists (`queue.ts:85`). Item `id = queue_<ts>_<rand6>` (`generateItemId, 372`).
- `dequeue(itemId)`: `Promise<boolean>` — removes by id, persists (`queue.ts:128`).
- `processQueue(config: ServerConfig)`: `Promise<QueueProcessResult>` (`queue.ts:144`):
 - Re-entrancy guard processing; no-op result if already processing or empty.
 - Builds a **fresh** `BobaAPIClient(config.url, config.requestTimeout)` + `qBitTorrentAdapter`.
 - Sorts a copy by priority `{high:0, normal:1, low:2}`.

- For each item: increments attempts, sets lastAttemptAt, calls sendQueuedItem, records {itemId, success, error}, marks successes for removal, delays 500ms between sends. Per-item exceptions captured into item.lastError + a failed result.
- Removes successful items, persists, returns {processed, succeeded, failed, remaining, results}.
- **Auth gap:** processQueue never authenticates the fresh client — queued private-tracker sends will fail unless the server needs no auth (Open Question).
- sendQueuedItem(item, adapter) (queue.ts:316): magnetUri → addTorrentFromMagnet; else torrentUrl → urlToFile(url) (fetch with credentials: "same-origin", blob → File named from the URL tail) → addTorrentFromFile.
- getSize(), getItems() (copy), clear().
- startAutoProcessing(config, intervalMs=60000) / stopAutoProcessing() — setInterval wrapper around processQueue (queue.ts:283). Background wires this at config.healthCheckInterval * 60000 ms when config.offlineQueue is on (background/index.ts:155-159).

2.4 Health checker HealthChecker (api/health.ts)

Thresholds (health.ts:28): healthy <2000ms, degraded <5000ms, MAX_FAILURES=2.

- checkServer(config): Promise<HealthCheckResult> — new BobaAPIClient, calls getVersion(), times it via performance.now(), derives status via determineStatus. On error: counts consecutive failures (1, or 2 if previous wasn't healthy), status unhealthy (≥MAX_FAILURES) else degraded. Caches into a Map<serverId,result> (health.ts:52). Result fields: serverId,url,status,version,responseTimeMs,authValid,error,checkedAt.
- checkAllServers(configs): Promise<HealthCheckResult[]> — sequential, re-entrancy guard, persists all results to STORAGE_KEYS.HEALTH keyed by serverId (health.ts:113, persistResults 271).
- testConnection(url): Promise<ConnectionTestResult> — unauthenticated version probe (timeout 5000ms HEALTH_CHECK); returns {success,url,version,error,responseTimeMs, testedAt} (health.ts:162).
- autoDiscover(): Promise<ConnectionTestResult[]> — probes http://localhost:<port> for **ports [8080, 7187, 7189]** (health.ts:201) i.e. qBittorrent WebUI, Boba FastAPI merge service, boba-jackett.
- getLastResult(serverId), getAllResults(), determineStatus(latency,success,failures).

2.5 qBittorrent adapter qBitTorrentAdapter (api/qbittorrent.ts)

Domain wrapper over the client. - `sendTorrent(torrent: DetectedTorrent, config): Promise<SendResult> — magnet → addMagnet(uri, config), torrent-file → addTorrentFile(url, config), else throws TorrentError. Returns`
`SendResult{success, torrent, error, response, completedAt}; never rethrows (normalizes errors to getUserMessage())`
`(qbittorrent.ts:46). - sendTorrents(torrents[], config): Promise<SendResult[]> — sequential with 250ms delay between sends (qbittorrent.ts:95-110). - addTorrentFile(url, config) (private) — fetch(url, {credentials:"same-origin"}) → blob → File("application/x-bittorrent") → addTorrentFromFile`
`(qbittorrent.ts:151). - buildAddOptions(config) — maps ServerConfig → add params: savepath, category, skip_checking, paused, autoTMM, contentLayout (mapped), upLimit/dlLimit (KiB→bytes ×1024) (qbittorrent.ts:190). mapContentLayout original→Original etc. - getClient() accessor.`

3. Parser rules (bencode / magnet / torrent-file + infohash)

3.1 Bencode parser/bencode.ts

Zero-dependency bencode using Uint8Array, TextEncoder/Decoder.
`BencodeValue = number | string | Uint8Array | BencodeValue[] | {[k]:BencodeValue}.`

encode(value): Uint8Array (bencode.ts:44) — concatenates parts. - Integer `i<N>e`; throws `ParseError` if not `Number.isInteger`. - String → UTF-8 bytes → `<len>:<bytes>`. Raw `Uint8Array` → `<len>:<bytes>`. - List `l...e` (bytes `0x6c/0x65`). - Dict `d...e`; **keys sorted lexicographically** via `Object.keys().sort()` (spec requirement — load-bearing for stable infohash) (bencode.ts:139-151).

decode(data, {encoding?: "utf-8"|"binary"}) (bencode.ts:165) — recursive descent on a `{data,pos}` cursor. Throws `ParseError` on **trailing data** after the top value. - `i` → integer: handles leading -, accumulates digits, requires `hasDigits`, throws on empty/ unterminated/invalid digit (bencode.ts:224). - digit `0x30-0x39` → byte-string `<len>:<bytes>`: validates length digits, bounds-checks `pos+length <= data.length` (throws "extends past end"); returns `Uint8Array` in binary mode, decoded UTF-8 string in utf-8 mode (bencode.ts:270). - `l` → list until `e`; `d` → dict until `e`. **Dict keys are always decoded as utf-8 strings** even when the value encoding is binary (bencode.ts:369) — important: it decodes keys utf-8 but

values may be binary. - Helpers `bytesToHex`, `hexToBytes`, and `sha1(data): Promise<string>` (Web Crypto `crypto.subtle.digest("SHA-1", ...) → hex`) (bencode.ts:397-427).

3.2 Magnet parser/magnet.ts

- `containsMagnetLink(text)` — fast `text.includes("magnet:?")` (magnet.ts:33).
- `findMagnetUri(text): string[]` — `MAGNET_REGEX (/magnet:\?xt=urn:btih:[a-fA-F0-9]{40}[\^s"'<>]*/gi)`, dedups case-insensitively, keeps only those passing `MAGNET_VALIDATION_REGEX` (anchored full-URI form) (magnet.ts:43).
- `extractInfohash(magnetUri): string|null` — `INFOHASH_REGEX (/xt=urn:btih:([a-fA-F0-9]{40})/i)`, lowercased, hex-validated (magnet.ts:72).
- `isValidHexInfohash (/^[a-fA-F0-9]{40}$/)`, `isValidBase32Infohash (/^[A-Z2-7]{32}$/)`.
- `base32ToHex(base32Hash)` — RFC4648 alphabet ABCDEFGHIJKLMNOPQRSTUVWXYZ234567; each char → 5 bits → grouped into 4-bit nibbles → hex (magnet.ts:110).
- **`parseMagnetUri(magnetUri, sourceElement=null): MagnetInfo`** (magnet.ts:159):
 1. requires `magnet:?` prefix (case-insensitive) else `ParseError`.
 2. hex infohash via `extractInfohash`; if none, tries base32 (`xt=urn:btih:([A-Z2-7]{32})/i`), converts to hex and **recurses** with the URI rewritten.
 3. Manually parses the query string (split on `&` OR `;`) into a `Map<string,string[]>` so **repeated keys** (multiple `tr`, `xt`) are preserved; values `decodeURIComponent-d`.
 4. Extracts params: **`xt`** (primary infohash, lowercased), **`dn`** (display name), **`tr`** (trackers[]), **`ws`** (web seeds[]), **`xl`** (exact length, `parseInt`), **`xs`** (exact source / .torrent URL), **`kt`** (keywords, split on `[+\s]+`), **`as`** (acceptable source), **`mt`** (manifest). Returns full `MagnetInfo` (see §types) with `detectedAt + sourceElement`. Wraps non-`ParseError` throws in `ParseError`.
- `buildMagnetUri(infohash, displayName?, trackers?)` — reconstructs `magnet:?xt=urn:btih:<hash>&dn=...&tr=...` (magnet.ts:291).
- `getMagnetDisplayName(magnet)` — falls back to `Torrent <first12hash>....`

3.3 Torrent file parser/torrent-file.ts + infohash

- **`parseTorrentFile(data: Uint8Array): Promise<ParsedTorrent>`** (torrent-file.ts:26):
 1. `decode(data)`; must be a dict, else `ParseError`.
 2. requires info sub-dict.
 3. **`Infohash = sha1(encode(infoDict))`** — re-bencodes ONLY the info dict and SHA-1s it (torrent-file.ts:46-48). This is

the canonical BitTorrent v1 infohash. (*Re-encode reorders dict keys lexicographically; for spec-compliant torrents this round-trips to the same bytes — but a torrent whose original info had non-sorted keys would hash differently. Caveat for the port.*)

4. requires name; extracts creation date, comment, created by (root), source (info), piece length, private (==1), counts pieces (pieces.length/20), trackers (extractTrackers), files (extractFiles), totalSize (sum of file lengths).
 - Returns ParsedTorrent (see §types).
- `parseTorrentFromUrl(url)` — `fetch(url, {credentials: "same-origin"})`, warns on unexpected content-type (expects `application/x-bittorrent` or `application/octet-stream`), then `parseTorrentFile(arrayBuffer)` (`torrent-file.ts:121`).
- `computeInfohash(data): Promise<string>` — lightweight: `decode → sha1(encode(info))` without full parse (`torrent-file.ts:159`).
- `extractTrackers(rootDict)` — announce (single) + announce-list (tiered nested arrays), dedup via Set, decodes Uint8Array trackers utf-8 (`torrent-file.ts:191`).
- `extractFiles(infoDict, defaultName)` — single-file (no files key) → one entry `{path:[name], length, fullPath:name}`; multi-file → iterate `files[]`, each `{path:[...], length, fullPath:path.join("/")}` (`torrent-file.ts:235`). `extractPath` decodes path components (utf-8) and filters empties.
- Type-safe extractors `getString/getStringOrNull/getNumber/getNumberOrNull` handle string-or-Uint8Array values.

Note: the `.torrent-file` PARSER (`parseTorrentFile/computeInfohash`) is fully built but is **never invoked** by the scanners, adapter, or background — `torrent-file` sends go straight to `qBittorrent` as raw File uploads (`addTorrentFromFile`). The parser is latent capability the Boba port could wire in for dedup/preview (Open Question).

4. Scanner architecture (orchestrator + link/text + site-db + batching)

4.1 BaseScanner (scanner/base.ts)

Abstract base. `ScannerOptions{scanShadowDom, maxElements, includeHidden, excludeSelector};` `DEFAULT_SCANNER_OPTIONS = {scanShadowDom:true, maxElements:10000, includeHidden:false, excludeSelector:"script,style,noscript,template,textarea"}` (`base.ts:35`). - `abstract scan(root?): Promise<DetectedTorrent[]>`, `getScannerId(): string`. - `isActive()`, `executeScan(fn)` — re-entrancy guard + timing + swallow-and-return-[] on error (`base.ts:106`). - `shouldIncludeElement(el)` — skips excluded selectors and (unless `includeHidden`) `display:none/visibility:hidden` via `getComputedStyle`

(base.ts:137). - querySelectorAllDeep(root, selector) — querySelectorAll + recursive **shadow-DOM** traversal (scanShadowDOM walks with TreeWalker, bounded by maxElements)
(base.ts:161). - createDetectedTorrent(type, displayName, magnet, torrentFile) — builds the unified DetectedTorrent; id from hashString(infohash ?? url ?? displayName); truncates display name to 80 chars (base.ts:235). hashString = djb2-ish
<base36hash>+<base36 now> — **id is time-salted, so the same torrent gets a different id each scan** (relevant to the orchestrator's dedup Map, which therefore dedups within a single scan pass only via the scanners' own seen sets, not across pages reliably) (base.ts:265).

4.2 LinkScanner (scanner/link-scanner.ts) — getId() => "link"

scan(root=document.body): 1. **Site-specific pass:** getSiteSelectors() resolves CSS selectors by domain (SITE_SELECTORS exact → base-domain → generic), runs each via querySelectorAllDeep (shadow-DOM aware), dedups by normalized href (seen Set). 2. **Generic fallback pass:** scanRoot.querySelectorAll("a[href]") over everything not yet seen. - For each href: magnet: → processMagnetLink (parse, displayName from magnet.displayName ?? el.textContent ?? Magnet <hash>); .torrent (via TORRENT_FILE_VALIDATION_REGEX /^https?:\/\/.+\.torrent(\?.*)?\$/i) → processTorrentLink (resolve relative→absolute URL, extract filename, sameOrigin flag) (link-scanner.ts:41).

4.3 TextScanner (scanner/text-scanner.ts) — getId() => "text"

scan(root=document.body) uses a TreeWalker(SHOW_TEXT) with an acceptNode filter that **skips** text inside script, style, noscript, template, textarea, code and text nodes shorter than 20 chars; bounded by maxElements. For each accepted node containing magnet:?, findMagnetUri extracts URIs, dedups, and processMagnetText parses + names them (display from magnet.displayName ?? parent[title] ?? parent.textContent ?? Magnet <hash>) (text-scanner.ts:39). Catches torrents pasted as plain text on forums.

4.4 ScannerOrchestrator (scanner/orchestrator.ts)

Coordinates scanners + MutationObserver.
OrchestratorOptions{enableLinkScanner, enableTextScanner, mutationDebounceMs, observeMutations}; defaults all-on, debounce DEBOUNCE_DELAYS.MUTATION=500. Constructor (orchestrator.ts:77): - creates its own TypedEventEmitter (or accepts one), - **per-site debounce:** getMutationDebounceForUrl(location.href) overrides the

default, - builds a `debounce(performScan, mutationDebounceMs)`, - `registerDefaultScanners()` → `LinkScanner` + `TextScanner`. - `start()` — initial `performScan()` + `setupMutationObserver()`. - `stop()` — cancel `debounce` + `disconnect observer`. - `scanNow(root?)` — manual immediate scan. - `getDetectedTorrents()/getDetectedCount()/clearDetected()/hasInitialScanCompleted()/isCurrentlyScanning()`. - `performScan(root?)` (`orchestrator.ts:226`): re-entrancy guard; emits **scan-started**; runs each registered scanner sequentially (only if `!scanner.isActive()`), **yieldToBrowser()** **between scanners** (cooperative, prevents main-thread block); aggregates; `mergeResults` dedups by id into `detectedMap` (emitting **torrent-detected** per new item); emits **scan-completed** (with counts + `durationMs`) or **scan-error**; returns `PageScanResult`. - `setupMutationObserver()` (`orchestrator.ts:354`): observes `document.body {childList:true, subtree:true, attributes:true, attributeFilter:["href"]}`. Only triggers the debounced rescan when a mutation is “relevant”: an added `<a>` / element containing `a[href^='magnet:'], a[href$='.torrent']`, an added text node containing `magnet:`, or an `href` attribute change to a `magnet/.torrent` value.

“Batching incl. tab-groups” — clarification: the reference code has **no chrome.tabGroups usage**. The batching that exists is (a) **per-scanner sequential batching with yieldToBrowser()** **cooperative yielding** in `performScan`, (b) the **processInChunks** helper in `utils.ts` (chunk size 50, yields between chunks — defined but unused by scanners), and (c) **per-tab result storage** in the background `tabTorrents: Map<tabId, PageScanResult>`. The plan’s “tab-group batching” must be designed fresh; the reference only batches DOM work within one tab. (Open Question.)

4.5 Site DB scanner/site-db.ts

A richer

`SiteConfig{domain,name,selectors,private,urlPatterns,dynamicContent, mutationDebounceMs}` registry (`SITES, site-db.ts:43`) for **15 sites**: `1337x.to`, `thepiratebay.org`, `rarbg.to`, `yts.mx`, `eztv.re`, `nyaa.si`, `limetorrents.lol`, `torrentgalaxy.to`, `fitgirl-repacks.site`, `rutracker.org` (private), `animetosho.org`, `demonoid.is`, `iptorrents.com` (private), `torrentleech.org` (private), `beyond-hd.me` (private), `passthepopcorn.me` (private). Each entry carries per-site `mutationDebounceMs` (400–800) and a `dynamicContent` flag. Exposed helpers: `getSiteConfig(url)` (exact domain → url-pattern → base-domain), `getSelectorsForUrl(url)`, `getMutationDebounceForUrl(url)` (default 500), `isKnownTorrentSite(url)`, `getSiteName(url)`, `listKnownSites()`.

Two overlapping selector tables: `LinkScanner` reads the **flat** `SITE_SELECTORS` map in `constants.ts` (covers ~21 domains incl. `rutracker`, `iptorrents`, `torrentleech`), while `site-db.ts`’s `SITES` map (15 richer entries) is only used for the per-site

debounce. rutracker.org/iptorrents.com/etc. are the privately-tracked sites Boba already supports — the port should reconcile these two tables.

5. Message protocol (content ↔ background ↔ popup/options)

Messages flow over `chrome.runtime.sendMessage / chrome.tabs.sendMessage`. `Envelope = ExtensionMessage{type: MessageType, payload?: Record<string, unknown>, requestId?}; reply = ExtensionMessageResponse{success, data?, error?}` (types/api.ts:450-486). The declared `MessageType` union and the actually-handled set differ slightly — both captured below.

5.1 Handled by background (background/index.ts:224 handleMessage)

type	payload in	returns	action
scan-result	{result: PageScanResult} (from content; tabId from sender)	{success}	stores into tabTorrents.s updateBadge(items.length
get-detected	{tabId?} (fallback sender.tab.id)	{success, data: {result}}	returns tabTorrents.get(
send-torrent	{ids:string[], tabId?}	{success, data: {results:SendResult[]}}	sendTorrents(tabId, ids)
get-config	—	{success, data: {config}}	loadConfig()
set-config	{config: ExtensionConfig}	{success}	persist + initializeApiC
health-check	—	{success, data: {results}}	healthChecker.checkAllSe
test-connection	{url}	{success, data: {result}}	healthChecker.testConne
auto-discover	—	{success, data: {results}}	healthChecker.autoDiscov 8080/7187/7189)
authenticate	{serverId, passphrase?}	{success, data: {authenticated}}	rebuilds client+auth, createCredentialsFromCon authenticate()
scan-page	{tabId?}	{success}	chrome.tabs.sendMessage(now"})
	—	{success}	

type	payload in	returns	action
open-dashboard			opens active server url (localhost:8080)
queue-status	—	{success, data:{size, items}}	offlineQueue.getSize()/g
queue-process	—	{success, data:{result}}	offlineQueue.processQueue
(default)	—	{success:false, error:"Unknown message type: ..."} type: ..."	

5.2 Handled by content script (content/index.ts:183 handleMessage)

type	payload in	returns	action
scan-now	—	{success}	orchestrator.scanNow
get-detected	—	{success, torrents:[{id,type,displayName,selected,sent}]}	simplified detected list (NOTE: a <i>different</i> shape than the background get detected)
toggle-selection	{id}	{success}	flips torrent.selected
get-scan-status	—	{success, isScanning, hasScanned, torrentCount}	orchestrator status
(default)	—	{success:false, error}	

5.3 Sent by content → background

- scan-result with {result: PageScanResult} — emitted from the orchestrator's scan-completed event handler (content/index.ts:243 sendScanResultToBackground).

5.4 Sent by popup → background / content (popup/popup.ts)

- get-detected {tabId} → background; falls back to chrome.tabs.sendMessage(tab.id, {type:"get-detected"}) to the content script directly (popup.ts:157-185).
- health-check → background (connection status dot) (popup.ts:198).
- scan-now → content script directly (Scan Page button) (popup.ts:110).

- `send-torrent {tabId, ids[]} → background` (Send button) (`popup.ts:378`).

5.5 Sent by options → background (options/options.ts)

- `set-config {config}` (on save) (`options.ts:102`).
- `auto-discover` (`options.ts:422`).
- `test-connection {url}` (`options.ts:507`).

5.6 Declared-but-unrouted message types

`MessageType` (`types/api.ts:450`) also declares `send-result`, `get-auth-state`, `health-result`, `show-notification`, `update-badge`, `torrent-detected`, `selection-change` — these have NO handler branch (the `background/content` switches don't cover them). They are aspirational / internal-event names overlapping the `EventMap`. `scan-page` is declared and IS handled by background but is only sent internally.

5.7 Internal event bus (shared/events.ts)

Not chrome-messaging — an in-process `TypedEventEmitter` with `EventMap`: `torrent-detected`, `scan-started`, `scan-completed`, `scan-error`, `send-started`, `send-completed`, `send-error`, `auth-state-changed`, `connection-status`, `config-changed`, `queue-updated`, `badge-update`, `notification`. The orchestrator emits scan events; the `HighlightManager` and `ContentScanner` subscribe. `on()` returns an `unsubscribe fn`; `once()`, `emit()`, `off()`, `listenerCount()`, `hasListeners()`.

6. Background service worker responsibilities (background/index.ts)

Central hub. Module-level state: `events` (emitter), `apiClient`, `authHandler`, `healthChecker`, `offlineQueue`, `tabTorrents`:
`Map<number, PageScanResult>`, `initialized`.

- **Lifecycle:** `chrome.runtime.onInstalled` (on install seeds `DEFAULT_CONFIG`), `chrome.runtime.onStartup`, and an immediate `initialize()` call (`background/index.ts:735-759`). Also an empty `onMessage` listener purely to keep the SW alive (`background/index.ts:754`).
- `initialize()` — loads config, inits logger + offline queue, sets up context menus, commands, message routing, storage listeners, alarms, and the API client; optional startup notification (`background/index.ts:80`).

- `initializeApiClient(config)` — finds `activeServerId` server, builds `BobaAPIClient` + `AuthHandler`; if `config.offlineQueue`, starts `offlineQueue.startAutoProcessing(server, healthCheckInterval*60000)`. Badge cleared/error accordingly (`background/index.ts:132`).
 - **Message routing** (`setupMessageRouting`) — async-safe `onMessage` returning `true`; delegates to `handleMessage` (§5.1).
 - **Torrent sending** `sendTorrents(tabId, ids)` (`background/index.ts:388`): loads active server, ensures `apiClient/authHandler`, `refreshIfNeeded(creds)` with `creds` decrypted using an **empty passphrase** "" (§11.4.10), pulls the tab's `PageScanResult`, filters items by id, runs `adapter.sendTorrents`, **enqueues failures** into the offline queue (if enabled), marks successes sent/`sendStatus:"success"`, updates badge, optionally notifies.
 - **Context menus** (`setupContextMenus`): `bobalink-send` (on link, url patterns `magnet:*,*://*/*.torrent,*://*/*.torrent?*`), `bobalink-scan` (`page/action`), `bobalink-dashboard` (`action/page`). `onClicked` → `handleContextSend` (sends a single link directly: `magnet` → `addTorrentFromMagnet`, `.torrent` → `fetch`→`File`→`addTorrentFromFile`, with category from active server) / `send scan-now` / `open dashboard` (`background/index.ts:481-585`).
 - **Keyboard commands** (`setupCommandListeners`, `chrome.commands`): `send-to-boba` (sends all unsent items of the active tab), `scan-page`, `open-dashboard` (`background/index.ts:594`).
 - **Alarms** (`setupAlarms`, `chrome.alarms`): **keepalive every 0.33 min (~20s)** (touches storage to keep SW alive); **health-check every 5 min** (runs `checkAllServers`) (`background/index.ts:637`).
 - **Badge & notifications**: `updateBadge(count,color)` (`chrome.action.setBadgeText` / `setBadgeBackgroundColor`, caps at 99); `showNotification(title,message,type)` (`chrome.notifications.create`, `icon/icon-128.png`, `error`→`priority 2`) (`background/index.ts:682-709`). Badge colors from `BADGE_COLORS` (`HEALTHY #4CAF50`, `DEGRADED #FF9800`, `ERROR #F44336`, `SCANNING #2196F3`, `DETECTED #9C27B0`, `DEFAULT #757575`).
 - **Storage listeners** (`setupStorageListeners`): on `CONFIG` change → `re-initializeApiClient` (`background/index.ts:718`).
-

7. Popup + options control / setting inventory

7.1 Popup (`popup/index.html` + `popup/popup.ts`)

UI elements / controls: - Header: brand icon + title; **connection status** dot (`#status-dot` `online/offline/warning`) + text (`#status-text`) driven by health-check results (online if any healthy, warning if degraded, offline otherwise; “No server”/warning banner

#connection-warning with an Open-Options link if no servers). - Toolbar buttons: **Select All** (#btn-select-all — selects all unsent), **Deselect** (#btn-deselect-all), **Refresh** (#btn-refresh — reloads list). - **Torrent list** (#torrent-list) — one row per DetectedTorrent: checkbox (disabled+checked if already sent), name (escaped+truncated 60), type icon/label, infohash first-16, “Sent” marker. Checkbox change toggles membership in a selectedIds Set. - **Empty state** (#empty-state) with a **Scan Page** button (#btn-scan-page → sends scan-now to the tab, shows a 2s progress overlay then reloads). - Footer: **selection info** (#selection-info “N selected”) + **Send to Boba** button (#btn-send, disabled when nothing selected → sendSelectedTorrents → send-torrent). - Progress overlay (#progress-overlay/#progress-text), error toast (3s auto-remove). - Open Options link (chrome.runtime.openOptionsPage()).

Reads: detected torrents per active tab, server health. Writes: selection (local), triggers sends + scans. No persisted settings written from the popup.

7.2 Options (options/index.html + options/options.ts)

Sidebar sections: **Servers, General, Advanced, About**. Save bar (#save-bar) with Discard/Save Changes; tracks hasChanges on any input/select change.

Servers section — server cards (name, url, active badge, auth-method badge; Set Active / Edit / Delete). **Add Server** opens a modal form: - #server-name, #server-url (validated isValidHttpUrl), #server-auth (none/cookie/api_key/basic — toggles .auth-fields-* visibility), - cookie/basic: #server-username + #server-password (or #server-basic-username/password), api_key: #server-apikey, - #server-category (default “BobaLink”), #server-savepath, - **Test Connection** (#btn-test-connection → test-connection), **Save Server** (submit). - On save (saveServerFromForm): encrypts secrets via encrypt(value, “bobaLink-extension”) (§11.4.10 fixed passphrase, options.ts:327), builds a full ServerConfig with hard-coded requestTimeout:15000, verifySsl:true, startPaused:false, skipHashCheck:false, contentLayout:“original”, autoTMM:false, uploadLimit:0, downloadLimit:0. First server becomes active. - **Auto Discovery card**: checkboxes (qBt 8080, FastAPI 7187, Go 7189 — display-only; actual ports hard-coded in health.autoDiscover), **Discover Servers** button → auto-discover → renders found servers with an **Add** button that prefills the modal.

General section toggles → ExtensionConfig fields: #setting-auto-scan→autoScan, #setting-highlight→highlightTorrents, #setting-highlight-style→highlightStyle (badge/border/glow), #setting-notifications→showNotifications, #setting-auto-send→autoSend, #setting-offline-queue→offlineQueue.

Advanced section: `#setting-health-interval→healthCheckInterval` (1–60), `#setting-max-history→maxHistoryItems` (10–500), `#setting-max-queue→maxOfflineQueueSize` (10–200), `#setting-request-timeout` (5–120s, displayed only — not written back to config), `#setting-debug→debugMode`. **Danger Zone:** Reset All (`#btn-reset` → restores `DEFAULT_CONFIG`).

About section: version, GitHub links, tech list.

Reads `STORAGE_KEYS.CONFIG`; on save persists config + sends set-config to background. `populateFormValues/readFormValues` bridge `form↔config`.

7.3 Assets / styles

- `assets/icon.svg` — 128×128 gradient magnet glyph (purple `#667eea`→`#764ba2` bg, pink accent tips). HTML references rasterized `icon-32/48/128.png` (must be produced at build); badge notification icon path `/icon-128.png`.
 - `content/styles.css` — three highlight styles, all `.bobalink-* + !important`: badge (absolute top-right gradient pill with icon+text), border (purple outline), glow (box shadow). `popup/styles.css` (dark theme, 380–420px) and `options/styles.css` are presentation only.
-

8. Defaults, keys & constants quick-reference (for the port)

- **Storage keys** (`constants.ts:213`, all `bobalink_*`): `CONFIG`, `AUTH_STATE`, `CREDENTIALS`, `DETECTED`, `HISTORY`, `QUEUE`, `HEALTH`, `KEY_MATERIAL`. (Actually written: `CONFIG`, `QUEUE`, `HEALTH`. `AUTH_STATE`/`CREDENTIALS`/`DETECTED`/`HISTORY`/`KEY_MATERIAL` declared but unused.)
- **DEFAULT_CONFIG** (`config.ts:140`): `schemaVersion:1`, `servers:[]`, `activeServerId:null`, `autoScan:true`, `autoScanDelay:2000`, `highlightTorrents:true`, `highlightStyle:"badge"`, `showNotifications:true`, `notificationSound:false`, `autoSend:false`, `maxHistoryItems:100`, `debugMode:false`, `healthCheckInterval:5`, `offlineQueue:true`, `maxOfflineQueueSize:50`, `showContextMenu:true`, `keyboardShortcuts:true`, `lastUpdated:0`, `encryptionKeyVersion:1`.
- **Ports:** FastAPI 7187, Go 7189, qBittorrent 8080 (`DEFAULT_PORTS`). Auto-discover probes `[8080,7187,7189]`. Open-dashboard fallback `http://localhost:8080`.
- **Timeouts** (`REQUEST_TIMEOUTS`): `DEFAULT` 15000, `HEALTH_CHECK` 5000, `AUTH` 10000, `ADD_TORRENT` 30000, `AUTO_DISCOVERY` 3000.

- **Retry** (RETRY_CONFIG): MAX_RETRIES 3, BASE 1000, MAX 30000, JITTER 0.3.
 - **Rate limit** (RATE_LIMIT): 10 req / 1000ms.
 - **Debounce** (DEBOUNCE_DELAYS): MUTATION 500, STORAGE_WRITE 250, AUTO_SCAN 1000, BADGE_UPDATE 300, NOTIFICATION 2000.
 - **Encryption** (ENCRYPTION): AES-GCM 256, IV 12B, salt 16B, PBKDF2 SHA-256 100000 iters.
-

9. Open questions / gaps the plan must resolve

1. **§11.4.10 — credential handling is broken-by-design.** The options page encrypts every secret with the **literal fixed passphrase "boba-link-extension"** (options.ts:327), and background.sendTorrents decrypts with an **empty passphrase ""** (index.ts:409). The per-encryption PBKDF2/AES-GCM machinery is sound, but a static or empty key makes the ciphertext trivially reversible and the two call-sites are inconsistent (an ""-decrypt cannot read a "boba-link-extension"-encrypted bundle). The Boba port **MUST** design a real key source (user master passphrase prompt, OS keychain, or delegate auth entirely to boba-jackett/the proxy which already owns encrypted credentials at /config/boba.db).
2. **Boba service integration is only port-deep.** Auto-discover knows ports 7187/7189 but the client only speaks the **qBittorrent WebUI v2 API**. There is NO code that talks to the Boba merge-search API (types/api.ts declares BobaServerInfo/BobaSearchResponse but nothing consumes them) or the proxy's tracker-cookie download path (7186) or boba-jackett (7189). The plan must define how the extension reaches Boba's own endpoints vs. raw qBt.
3. **set-cookie parsing for SID.** client.login parses SID from the set-cookie response header, which browsers normally strip from fetch; works incidentally via the cookie jar + credentials:"include". Brittle across CORS / cross-origin proxy setups.
4. **No CORS / host-permissions story.** All fetches go cross-origin to localhost:8080/7187/ 7189; MV3 needs host_permissions + qBittorrent must allow the extension origin (or the download-proxy must proxy). The manifest itself was not in scope — must be authored.
5. **Offline queue never authenticates.** processQueue builds a fresh client and never logs in, so retries against an authed server will 403/fail loop. Needs auth wiring.
6. **.torrent parser is dead code.** parseTorrentFile/computeInfohash (full bencode + SHA-1 infohash) are implemented and tested-shaped but never called — sends upload the raw file. Port

could wire it for local dedup / infohash-based dedup against Boba.

7. **Two divergent site tables.** `constants.SITE_SELECTORS` (used by LinkScanner) vs `site-db.SITES` (used only for debounce) overlap and disagree; the private trackers Boba actually supports (rutracker/iptorrents/etc.) are split across both. Reconcile.
8. **“Tab-group batching” does not exist** in the reference. Only intra-tab `yieldToBrowser` batching + per-tab `tabTorrents` map. The plan’s tab-group batch behavior is net-new design.
9. **Time-salted detection ids.** `BaseScanner.hashString` appends `Date.now()`, so a torrent’s id changes every scan; cross-scan / cross-tab dedup relies on per-pass seen sets and is unreliable. Port should use infohash-derived stable ids.
10. **Two different get-detected response shapes** (background returns `{data:{result}}`, content returns `{torrents:[...]}`) — the popup tries both; a unified contract is cleaner.
11. **Unrouted declared message types** (`send-result`, `get-auth-state`, `health-result`, `show-notification`, `update-badge`, `torrent-detected`, `selection-change`) — decide whether to implement or drop.
12. **Manifest, commands, and icon rasterization** (`manifest.json`, `chrome.commands` definitions for `send-to-boba/scan-page/open-dashboard`, PNG icon sizes 16/32/48/128) are referenced by code but not present in the read scope — must be produced for the port.